1. Git & GitHub

🔹 GitHub Issues

GitHub Issues is a feature used to:

- Ask questions about a project
- Suggest improvements
- Track tasks or changes (not only bugs)

☑ Key Features

- Issues can have **labels** (bug, feature, etc.)
- Can be **assigned** to individuals
- Can be linked to **milestones** (project goals)
- Each issue supports **discussion via comments**
- Users can **subscribe** to get updates
- Supports **Markdown formatting**

🔹 Best Practices for Issues

- Always write **clear and descriptive issues**
- Check existing issues before creating a new one
- Communicate properly with project maintainers
- Clearly explain what change or feature you want

🔹 Git Workflow (Best Practices)

Branching

- Never work directly on `main/master`
- Always create a **separate branch (topic branch)**
- Use **clear and meaningful branch names**

Commits

- Make **small and frequent commits**
- Use **clear commit messages**
- Keep messages meaningful and descriptive
- Update `README.md` if needed

2. Database Theory

🔹 What is a Database?

A database is:

A structured collection of logically related data designed to meet the information needs of an organization.

🔹 Importance of Databases

- **Data Storage** → Store large structured data
- **Data Analysis** → Generate reports and insights
- **Record Keeping** → Manage records (customers, transactions)
- **Web Applications** → Provide dynamic content and user management

🔹 Properties of an Ideal Database

1. **Integrity** → Data should be accurate and consistent
2. **Availability** → Data should be accessible when needed
3. **Security** → Protection from unauthorized access
4. **Independence** → Separate from application logic
5. **Concurrency** → Multiple users can access data simultaneously

3. Types of Databases

1. Relational Databases (SQL)

- Data stored in **tables (rows & columns)**

2. NoSQL Databases

- Handle **unstructured/semi-structured data**

3. Column Databases

- Store data in **columns**
- Used for analytics and warehousing

4. Graph Databases

- Store relationships (networks, connections)

5. Key-Value Databases

- Store data as **key-value pairs**

4. DBMS (Database Management System)

🔹 Definition

A DBMS is software that:

- Manages databases
- Acts as a bridge between users and data

🔹 Core Functions of DBMS

- **Data Management** → Store, retrieve, update data
- **Integrity** → Maintain data accuracy
- **Concurrency** → Multi-user access
- **Transactions** → All-or-nothing operations
- **Security** → Restrict unauthorized access
- **Utilities** → Backup, import/export, logging

5. Database Keys

Keys uniquely identify records and maintain relationships.

🔹 Types of Keys

- **Super Key** → Uniquely identifies rows (may have extra attributes)
- **Candidate Key** → Minimal super key
- **Primary Key** → Unique, no NULL values
- **Alternate Key** → Candidate key not selected as primary
- **Composite Key** → Combination of multiple columns
- **Surrogate Key** → Artificial key (e.g., auto ID)
- **Foreign Key** → Links two tables

6. Cardinality of Relationships

Cardinality defines how entities relate to each other.

Examples:

- Person → Driving License (1:1)
- Student → Branch (Many:1)
- Restaurant → Orders (1:Many)
- Student → Courses (Many:Many)

7. Drawbacks of Databases

- **Complexity** → Difficult to set up and maintain
- **Cost** → High infrastructure cost
- **Scalability Issues** → Hard to manage large data
- **Data Integrity Challenges**
- **Security Risks**
- **Data Migration Issues**
- **Lack of Flexibility**